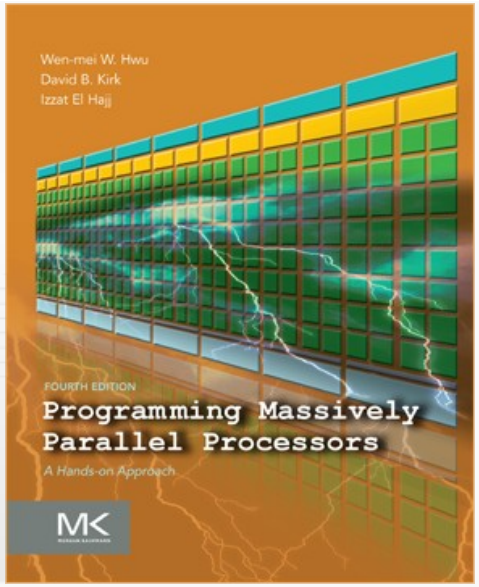
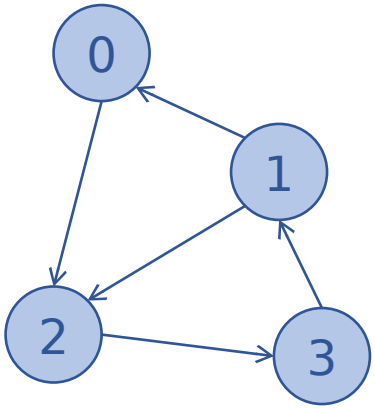
Programming Massively Parallel Processors

A Hands-on Approach

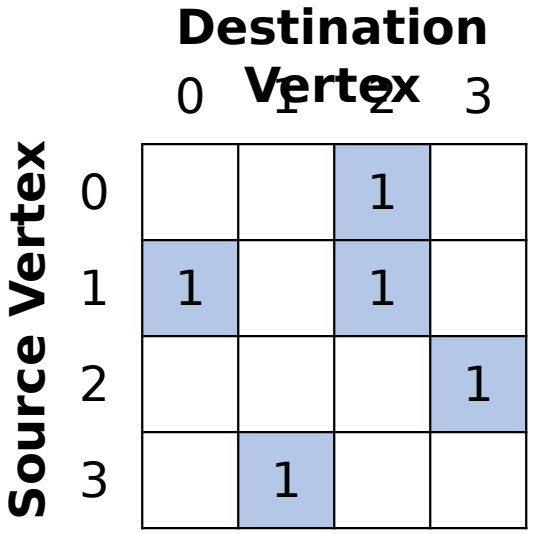
CHAPTER 15 (PART 1) **› Graph Traversal**



Representing Graphs



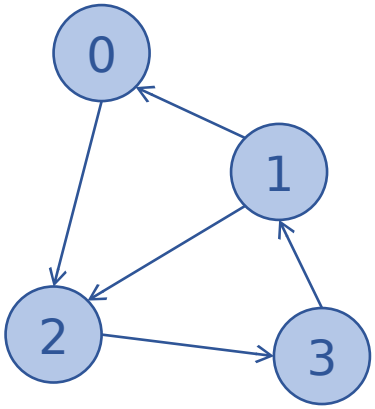
Graph



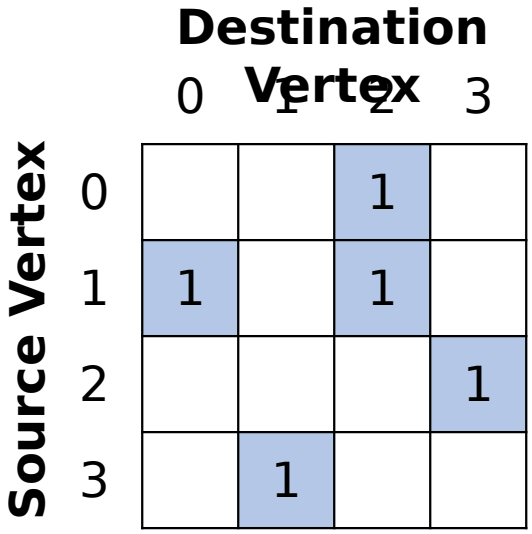
Adjacency Matrix

- Graphs can be represented with adjacency matrices
  - Adjacency matrix is usually very sparse
    - Can use the same storage formats for sparse matrices
  - We will assume unweighted graphs
    - Nonzeros all ones so no need to store their values

COO, CSR, and CSC Representation



Graph



Adjacency Matrix

CSR

srcPtrs	0	1	3	4	5
dst	2	0	2	3	1

CSC

dstPtrs	0	1	2	4	5
src	1	3	0	1	2

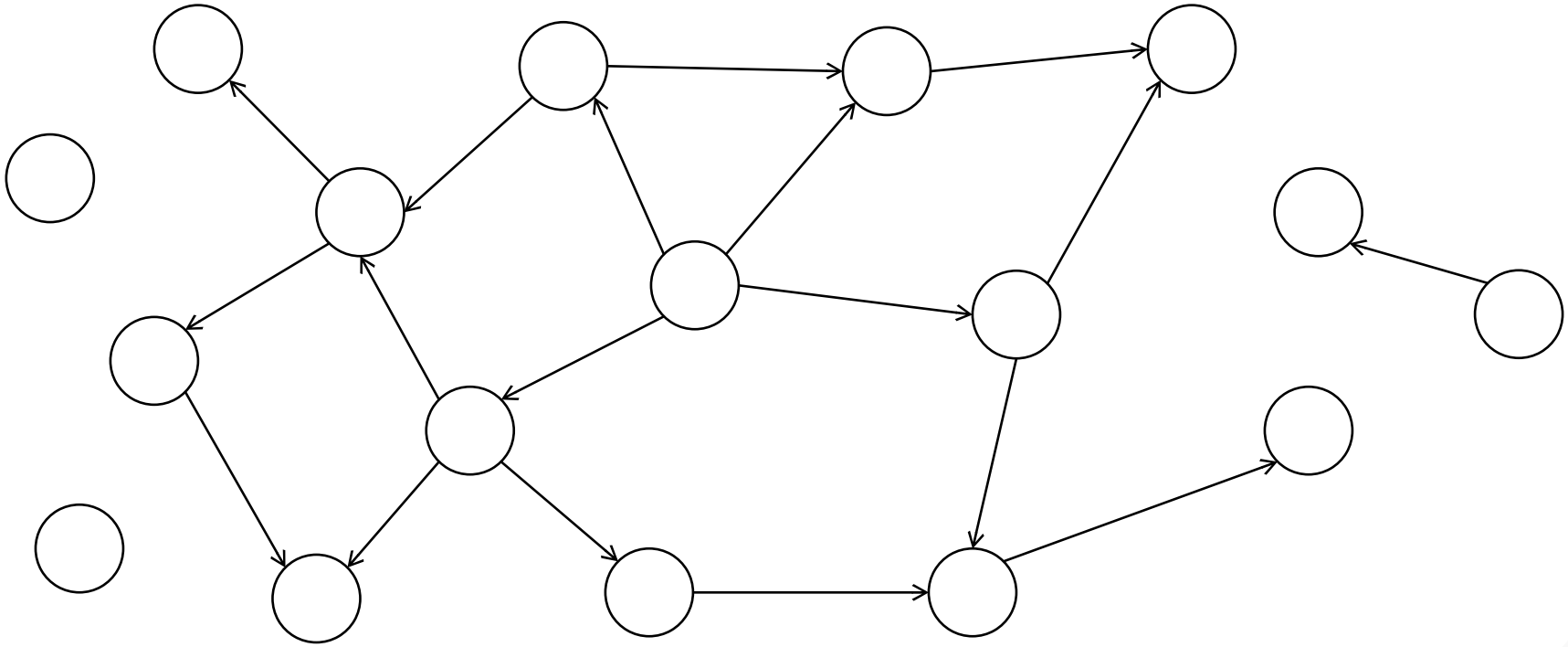
COO

src	0	1	1	2	3
dst	2	0	2	3	1

Approaches to Parallelizing Graph Traversal

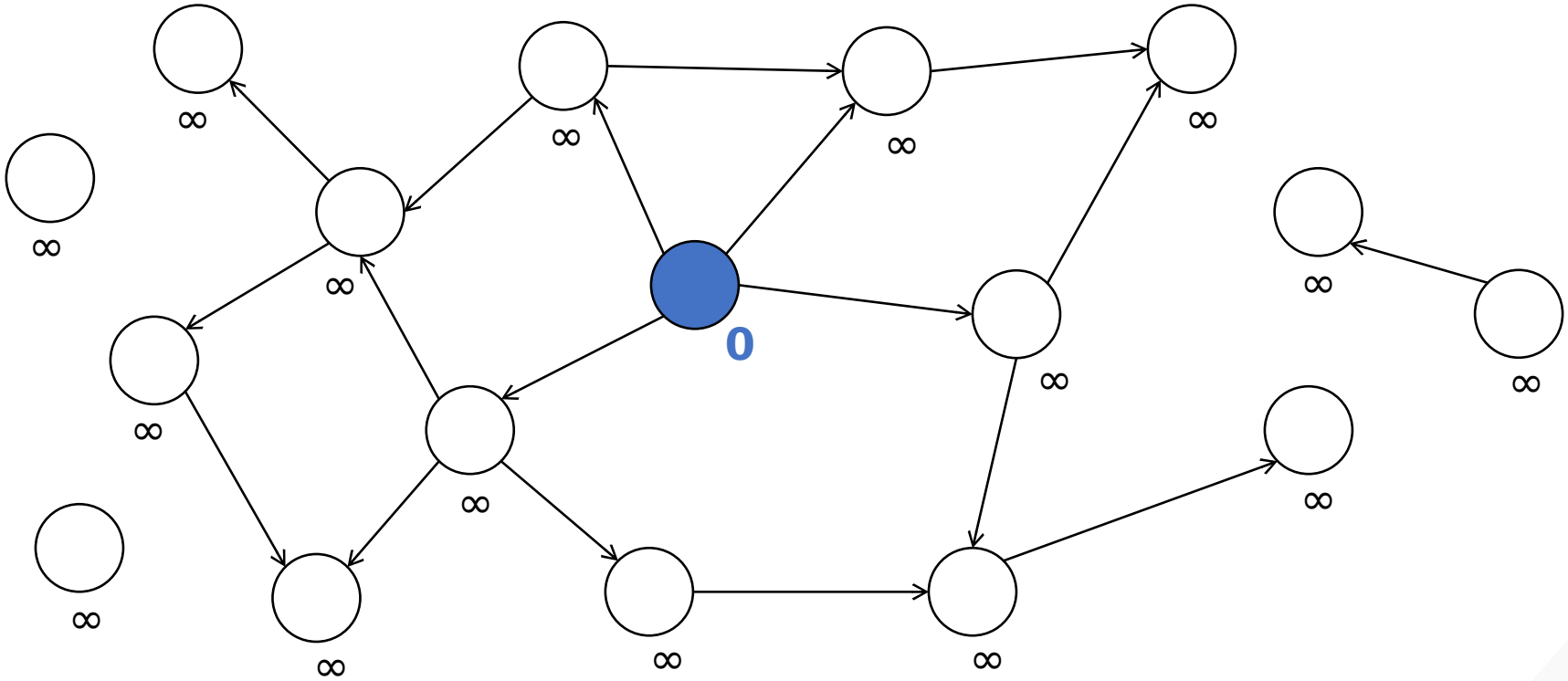
- Vertex-centric
  - Assign a thread to do something for each vertex
  - Typically use CSR/CSC
    - Given a vertex, easy to find all its incoming or outgoing edges
    - Might also use other formats (e.g., ELL, JDS) as optimization
- Edge-centric
  - Assign a thread to do something for each edge
  - Typically use COO
    - Given an edge, easy to find its source and destination vertices
- Hybrid
  - Example: Given an edge, need neighbors of the source vertices and neighbors of the destination vertices
  - Use both COO and CSR/CSC

Breadth First Search (BFS)



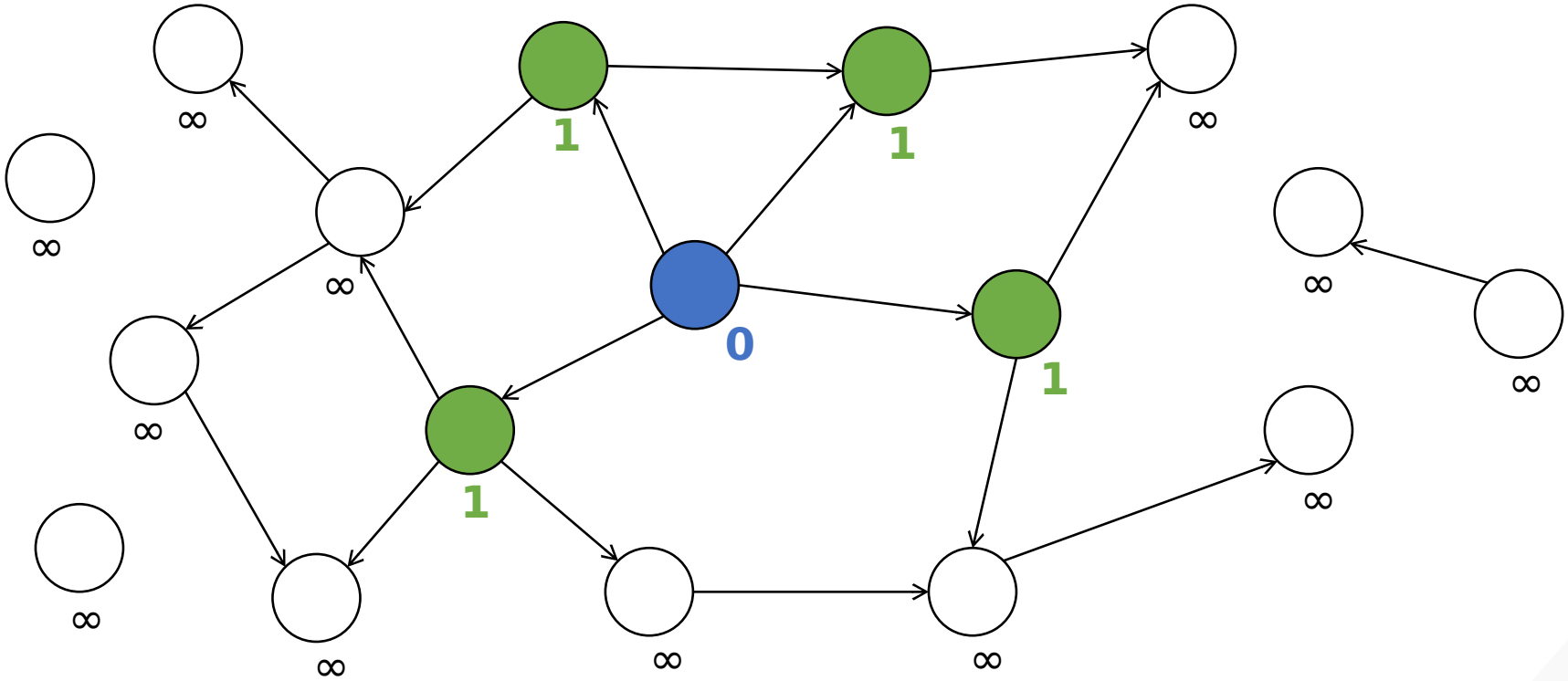
Objective: find the distance (level) of each vertex from some source vertex

Breadth First Search (BFS)



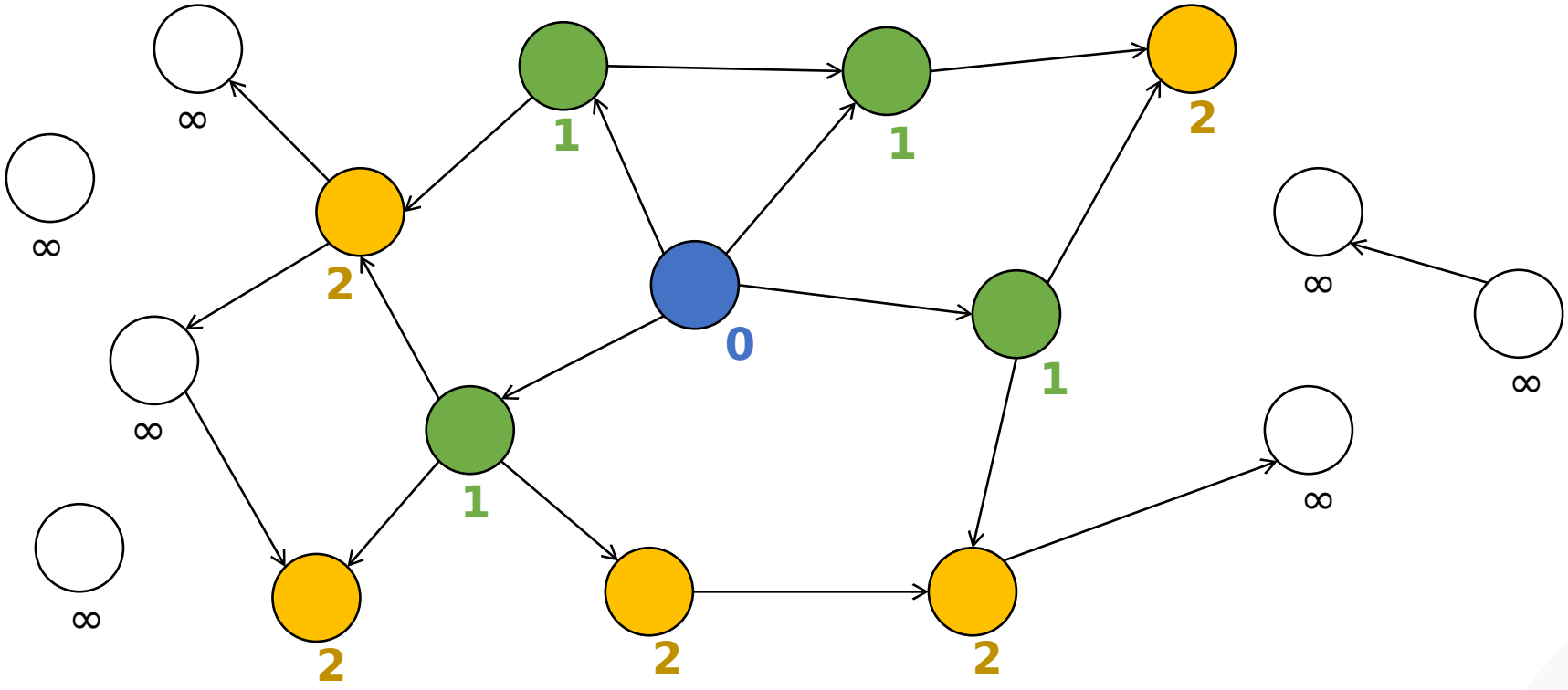
Objective: find the distance (level) of each vertex from some source vertex

Breadth First Search (BFS)



Objective: find the distance (level) of each vertex from some source vertex

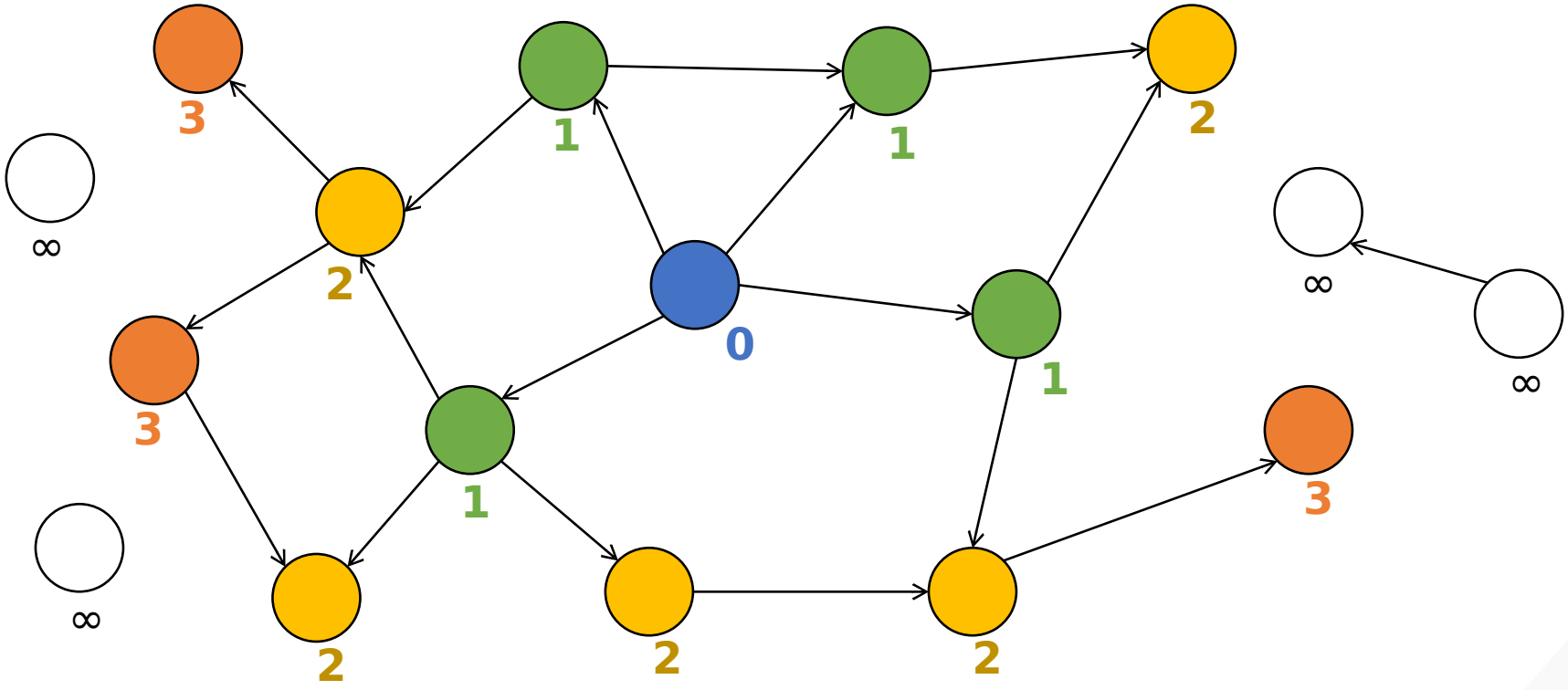
Breadth First Search (BFS)



Level 2

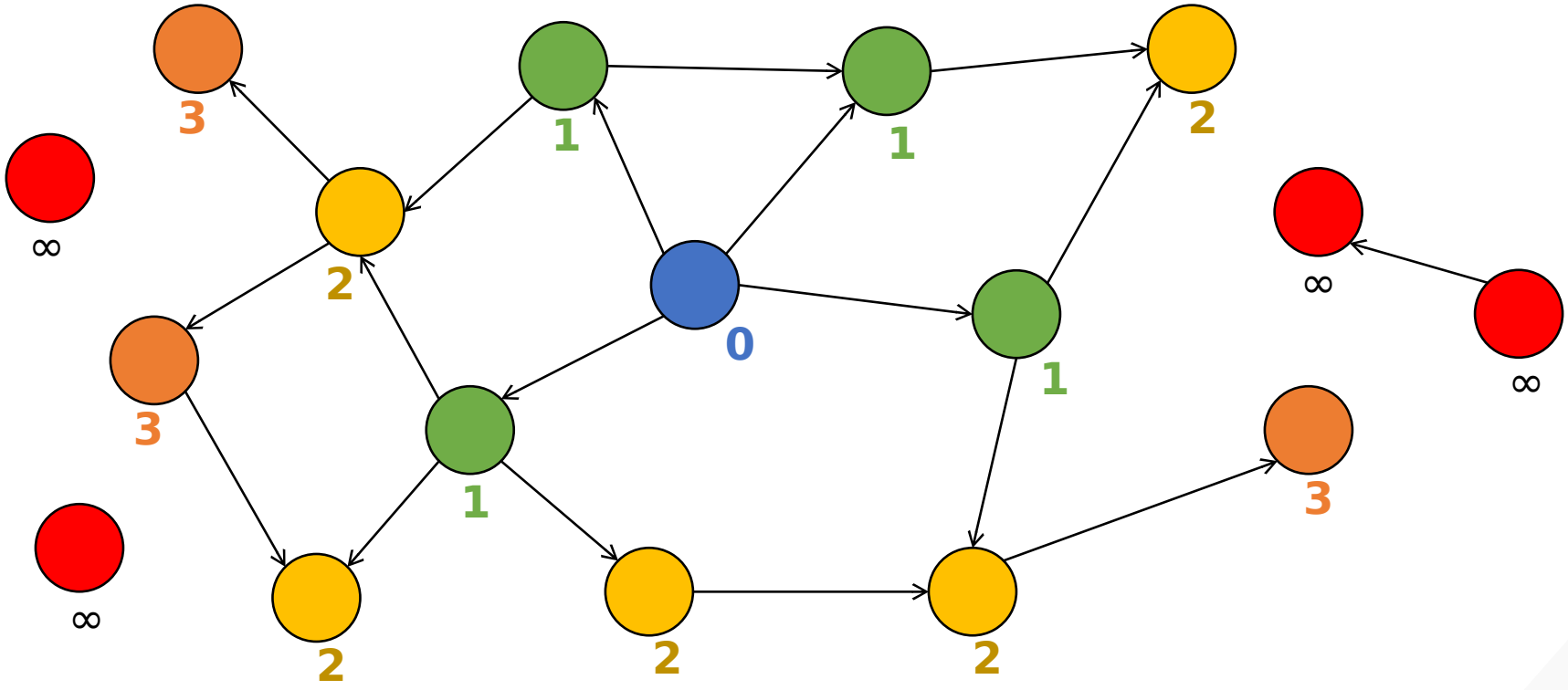
Objective: find the distance (level) of each vertex from some source vertex

Breadth First Search (BFS)



Objective: find the distance (level) of each vertex from some source vertex

Breadth First Search (BFS)



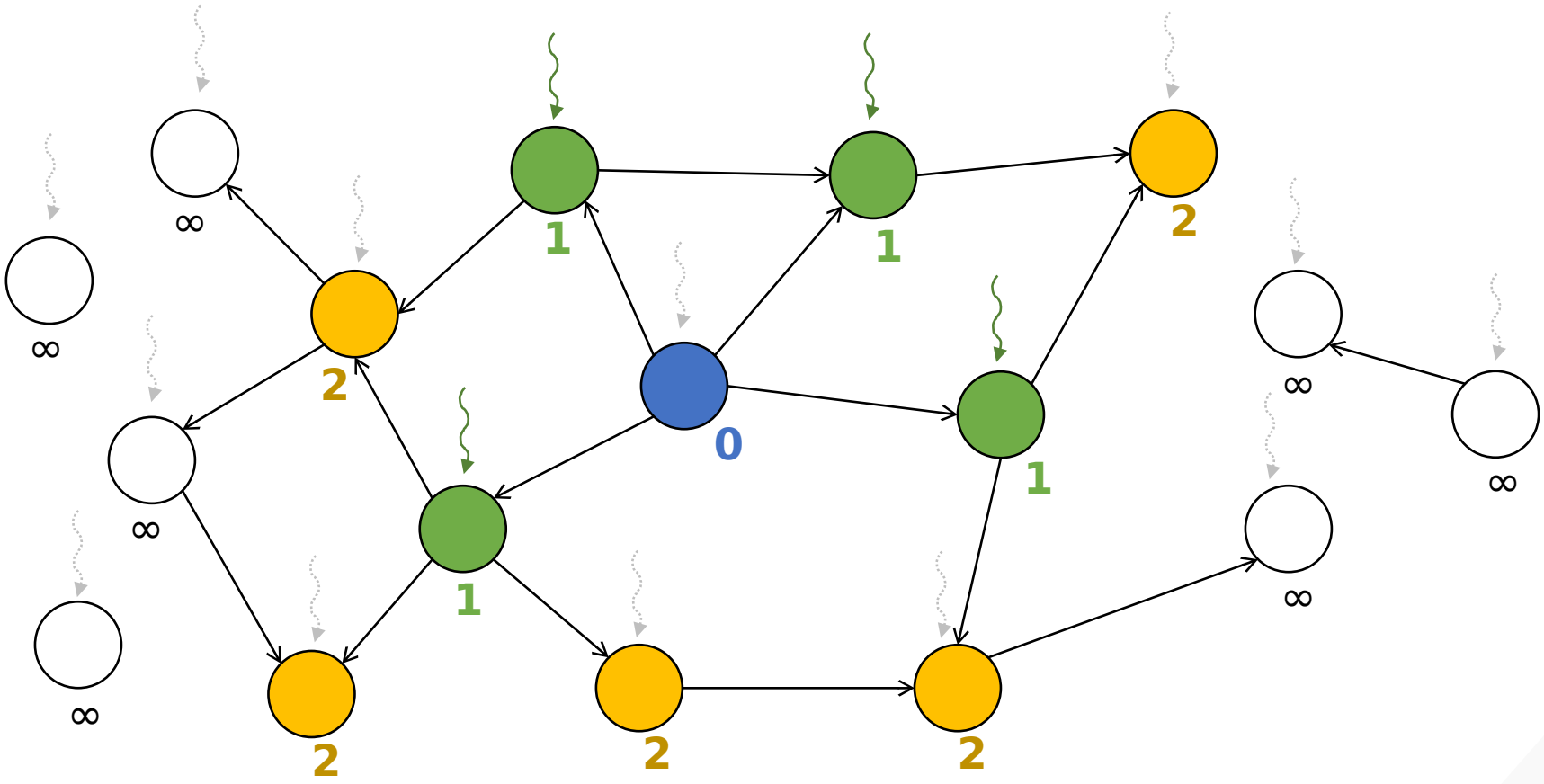
Unreachable

Objective: find the distance (level) of each vertex from some source vertex

Approaches to Parallelizing BFS

- Vertex-centric (two versions)
  - **Push:** For every vertex, if it was in the previous level, add all its unvisited outgoing neighbors to the current level
    - i.e., a vertex pushes information to its outgoing neighbors

Vertex-Centric BFS (Push)



Level 1 => Level 2

ads whose vertices are in **level 1** mark their unvisited neighbors as being in **lev**

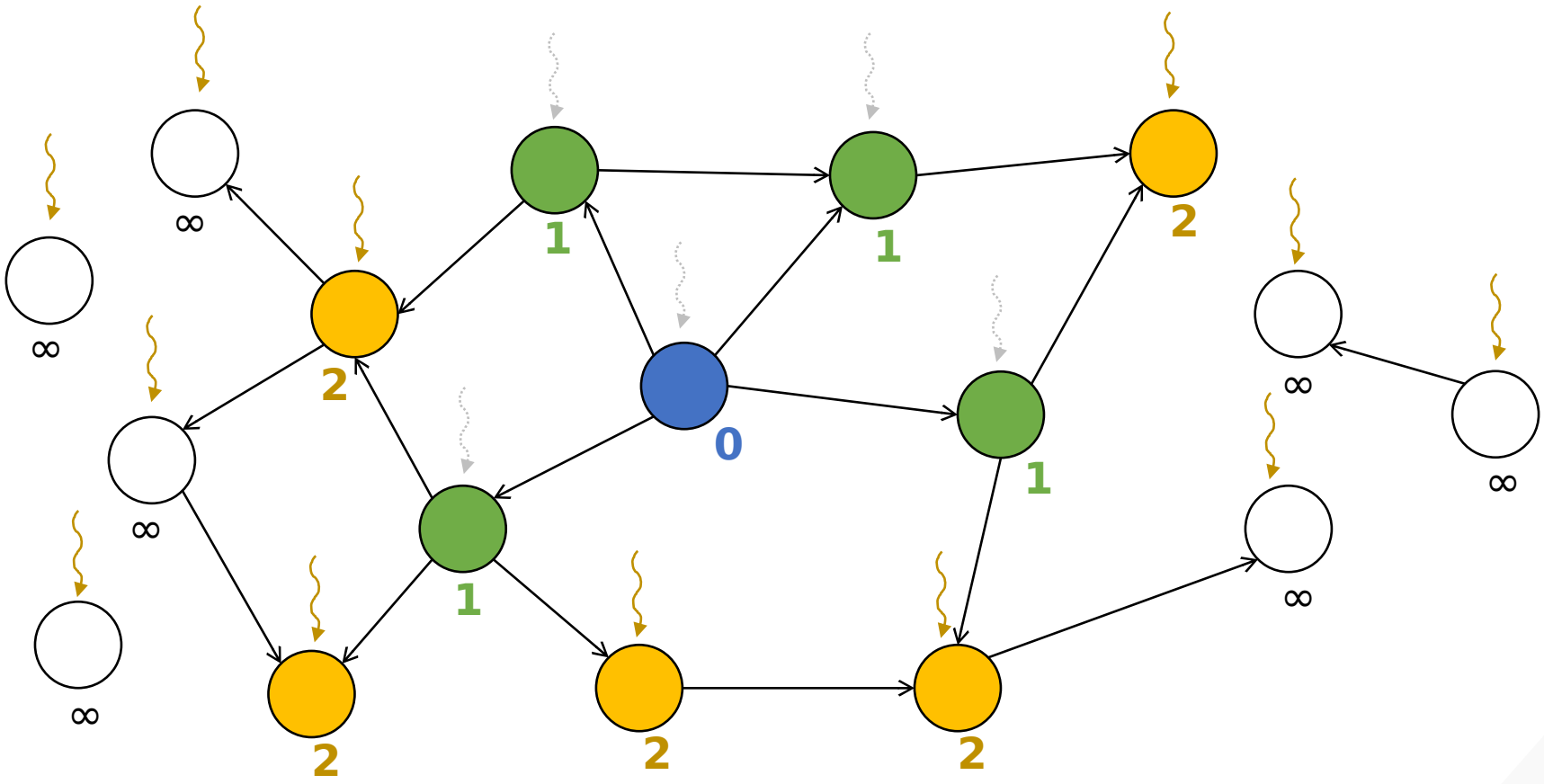
Vertex-Centric BFS Code (Push)

```
__global__ void bfs_kernel(CSRGraph csrGraph, unsigned int* level, unsigned int* newVertexVisited,
                                                                    unsigned int currLevel) {
    unsigned int vertex = blockIdx.x*blockDim.x + threadIdx.x;
    if(vertex < csrGraph.numVertices) {
        if(level[vertex] == currLevel - 1) {
            for(unsigned int edge = csrGraph.srcPtrs[vertex]; edge < csrGraph.srcPtrs[vertex + 1]; ++edge) {
                unsigned int neighbor = csrGraph.dst[edge];
                if(level[neighbor] == UINT_MAX) { // Neighbor not previously visited
                    level[neighbor] = currLevel;
                    *newVertexVisited = 1;
                }
            }
        }
    }
}
```

Approaches to Parallelizing BFS

- Vertex-centric (two versions)
  - **Push:** For every vertex, if it was in the previous level, add all its unvisited outgoing neighbors to the current level
    - i.e., a vertex pushes information to its outgoing neighbors
  - **Pull:** For every vertex, if it has not been visited, if any of its incoming neighbors are in the previous level, add it to the current level
    - i.e., a vertex pulls information from its incoming neighbors

Vertex-Centric BFS (Pull)



Level 1 => Level 2

Threads whose vertices are unvisited mark their vertices as being in **level 2** if any of their neighbors are in **level 1**

Vertex-Centric BFS Code (Pull)

```
__global__ void bfs_kernel(CSCGraph cscGraph, unsigned int* level, unsigned int* newVertexVisited,
                                                                    unsigned int currLevel) {
    unsigned int vertex = blockIdx.x*blockDim.x + threadIdx.x;
    if(vertex < cscGraph.numVertices) {
        if(level[vertex] == UINT_MAX) { // Vertex not previously visited
            for(unsigned int edge = cscGraph.dstPtrs[vertex]; edge < cscGraph.dstPtrs[vertex + 1]; ++edge) {
                unsigned int neighbor = cscGraph.src[edge];
                if(level[neighbor] == currLevel - 1) {
                    level[vertex] = currLevel;
                    *newVertexVisited = 1;
                    break;
                }
            }
        }
    }
}
```

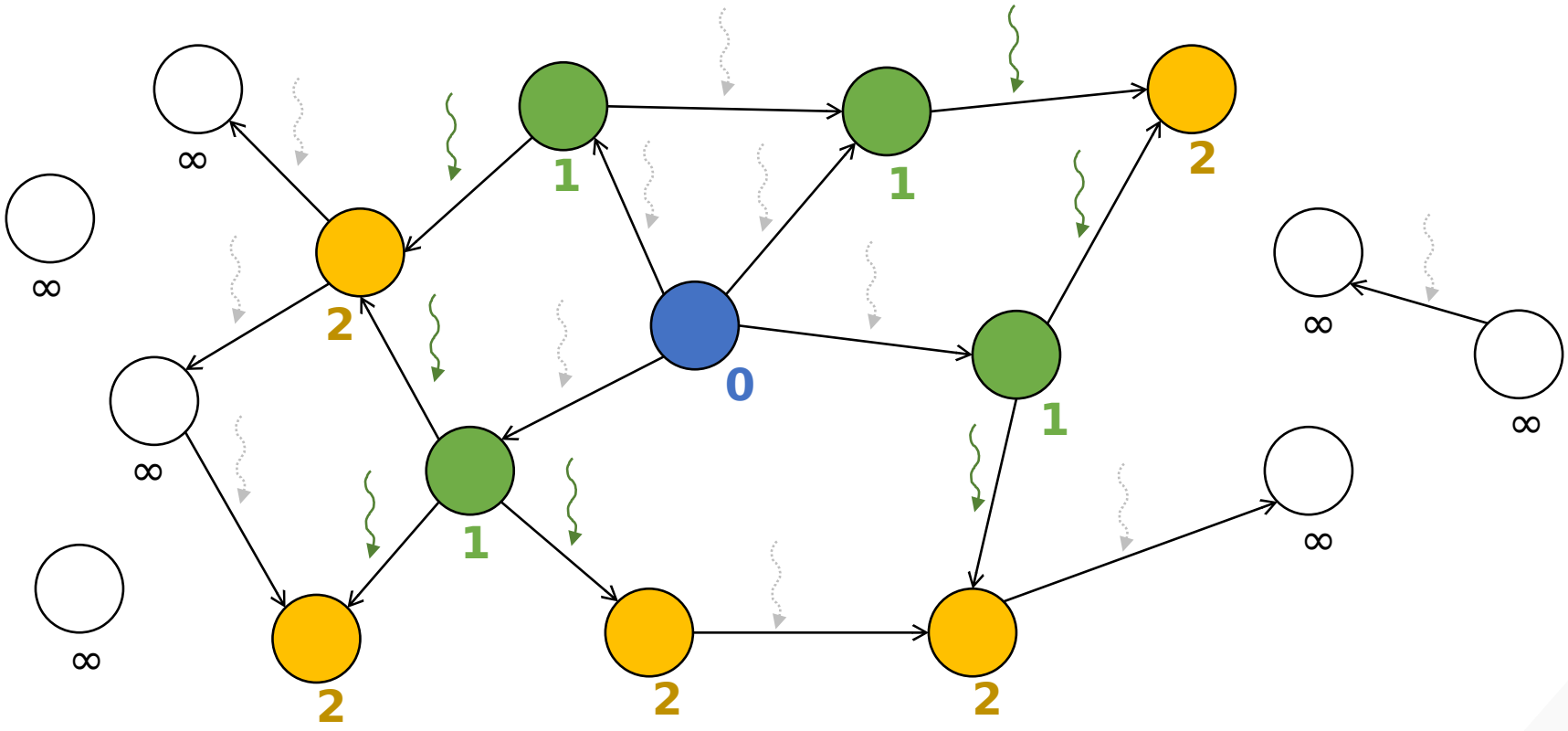
Approaches to Parallelizing BFS

- Vertex-centric (two versions)
  - **Push:** For every vertex, if it was in the previous level, add all its unvisited outgoing neighbors to the current level
    - i.e., a vertex pushes information to its outgoing neighbors
  - **Pull:** For every vertex, if it has not been visited, if any of its incoming neighbors are in the previous level, add it to the current level
    - i.e., a vertex pulls information from its incoming neighbors
  - **Direction-optimized:** Start with push then switch to pull
    - Pull is inefficient at the beginning because most vertices will search all neighbors and not find any that are in the previous level

Approaches to Parallelizing BFS

- Vertex-centric (two versions)
  - **Push:** For every vertex, if it was in the previous level, add all its unvisited outgoing neighbors to the current level
    - i.e., a vertex pushes information to its outgoing neighbors
  - **Pull:** For every vertex, if it has not been visited, if any of its incoming neighbors are in the previous level, add it to the current level
    - i.e., a vertex pulls information from its incoming neighbors
  - **Direction-optimized:** Start with push then switch to pull
    - Pull is inefficient at the beginning because most vertices will search all neighbors and not find any that are in the previous level
- Edge-centric
  - For every edge, if its source vertex was in the previous level, add its destination vertex to the current level

Edge-Centric BFS



Level 1 => Level 2

Threads whose edge's source vertex is in **level 1** and whose edge's destination vertex is unvisited mark their edge's destination vertex as being in **level 2**

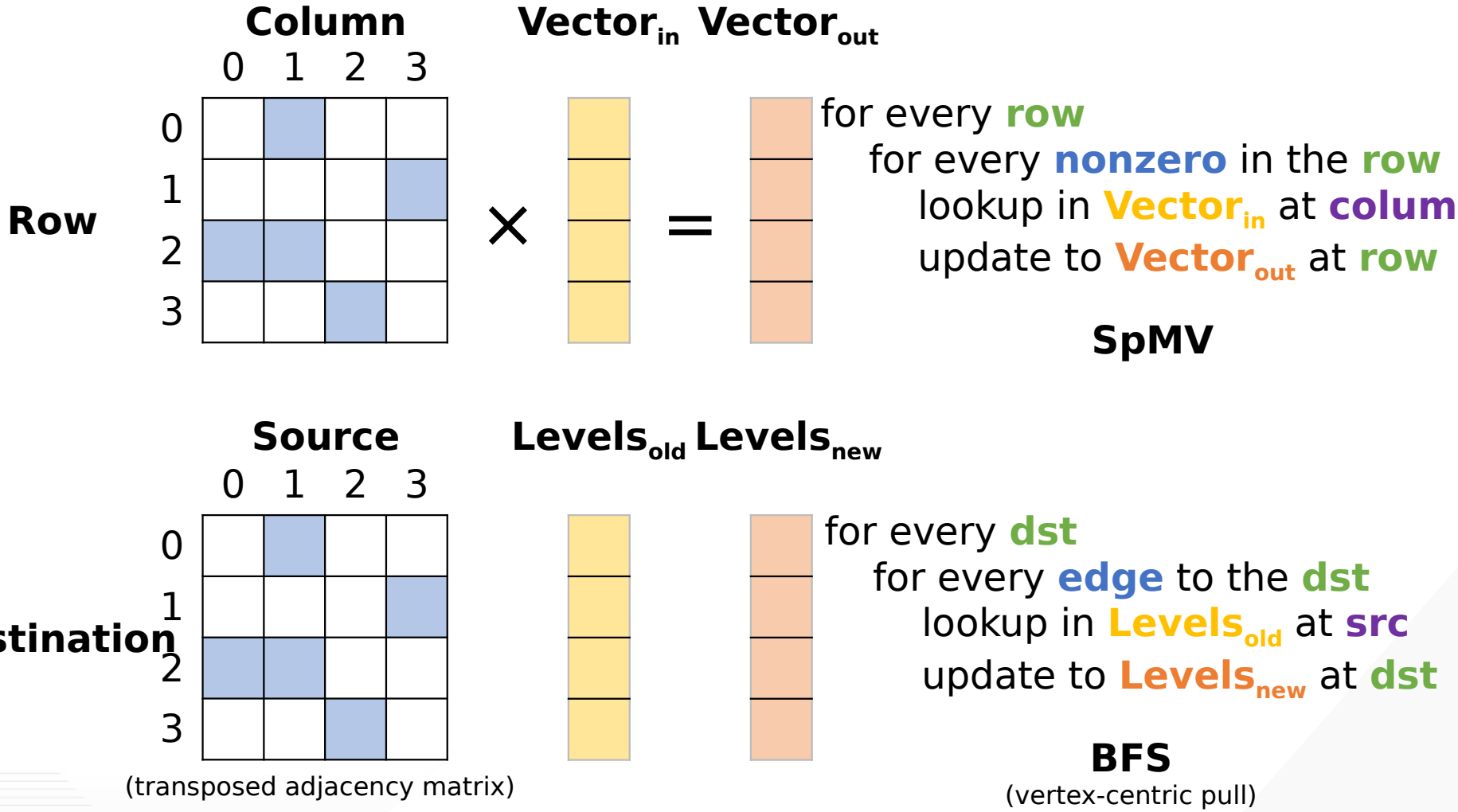
Edge-Centric BFS Code

```
__global__ void bfs_kernel(COOGraph cooGraph, unsigned int* level, unsigned int* newVertexVisited,
                                                                    unsigned int currLevel) {
    unsigned int edge = blockIdx.x*blockDim.x + threadIdx.x;
    if(edge < cooGraph.numEdges) {
        unsigned int vertex = cooGraph.src[edge];
        unsigned int neighbor = cooGraph.dst[edge];
        if(level[vertex] == currLevel - 1) {
            if(level[neighbor] == UINT_MAX) { // Destination vertex not previously visited
                level[neighbor] = currLevel;
                *newVertexVisited = 1;
            }
        }
    }
}
```

Dataset Implications

- The best parallelization approach depends on the structure of the graph
- The vertex-centric pull and the edge-centric approaches are better on high-degree graphs
  - e.g., social network graphs with celebrities
  - Better at dealing with load imbalance
- The vertex-centric push approach is better on low-degree graphs
  - e.g., map of roads in a geographical area
  - Only promising threads will iterate through neighbors

Similarity Between BFS and SpMV



ex-centric push and edge-centric also correspond to other ways of performing S

Linear Algebraic Formulation of Graph Problems

- With a few tweaks, BFS can be formulated exactly as SpMV

- Many graph problems can be formulated in terms of sparse linear algebra computations
  - Advantage: leverage mature and well-optimized parallel libraries for high performance sparse linear algebra
  - Disadvantage: not always the most efficient way to solve the problem

References

- Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2022.